

**Department of Computer Science and
Engineering**

Assignment Questions

Course Code & Name:

CSA0603 –Design and Analysis of Algorithms

Name: AKSHARAA BALASUBRAMANIAN

Reg No: 192311278

Branch: Computer Science

ANALYTICAL STUDY AND IMPLEMENTATION OF MERGE SORT

1.1 Problem Statement

Sorting is an important operation in computer science used to arrange data in a particular order. For large datasets, inefficient sorting algorithms can require significant computational time. Merge Sort is a divide-and-conquer sorting algorithm that divides an input array into smaller subarrays, recursively sorts them, and finally merges the sorted subarrays.

For this study, the given dataset is:

$$A = \{42, 17, 8, 93, 26, 55, 11, 68, 34, 5, 79, 21\}$$

The objective is to sort the elements in ascending order using Merge Sort, develop its pseudocode and implementation, analyze its time and space complexity, and compare its performance with the brute-force Selection Sort mentioned in the assignment. The assignment specifically asks students to compare Selection Sort with an efficient divide-and-conquer algorithm such as Merge Sort.

Input

An unsorted array of n numerical elements.

Output

The elements arranged in ascending order.

Constraints

- Elements may be integers or real numbers.
- The algorithm should correctly sort all input elements.
- The algorithm should work for different input sizes.
- Performance should be evaluated using execution time and operation counts.

2. Why Merge Sort?

Merge Sort is selected because it uses the Divide and Conquer strategy.

It works in three stages:

1. Divide – Divide the array into two halves.
2. Conquer – Recursively sort both halves.
3. Combine – Merge the two sorted halves.

Unlike Selection Sort, whose expected running time is $\Theta(n^2)$, Merge Sort has a running time of:

$$\Theta(n \log n)$$

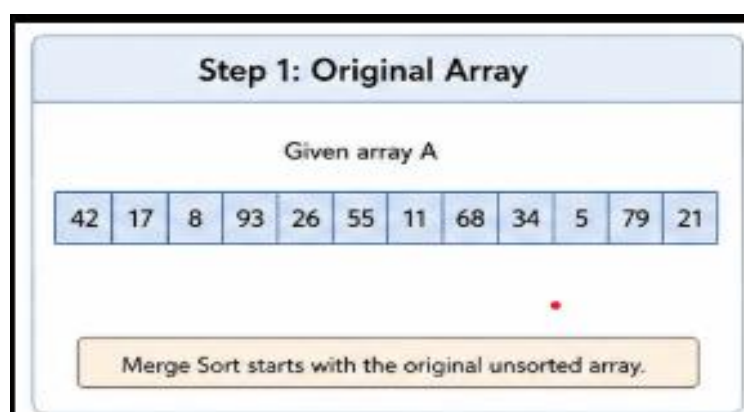
Therefore, Merge Sort is more suitable for large datasets.

3. Working of Merge Sort

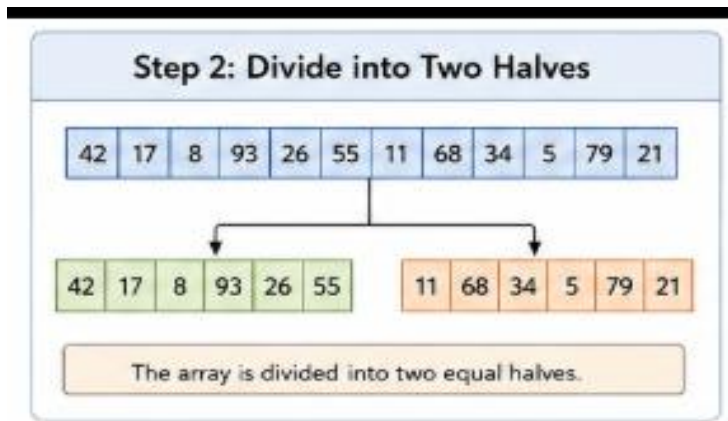
For the given array:

{42 ,17, 8, 93, 26 ,55 ,11 ,68 ,34, 5 ,79 ,21}

Step 1 – Divide

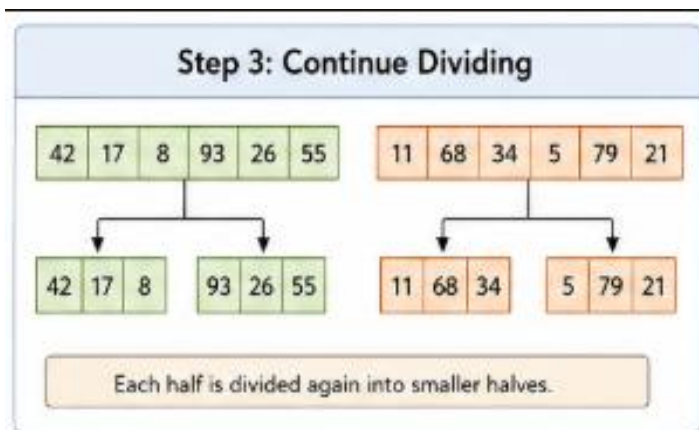


Step 2 – Continue Dividing



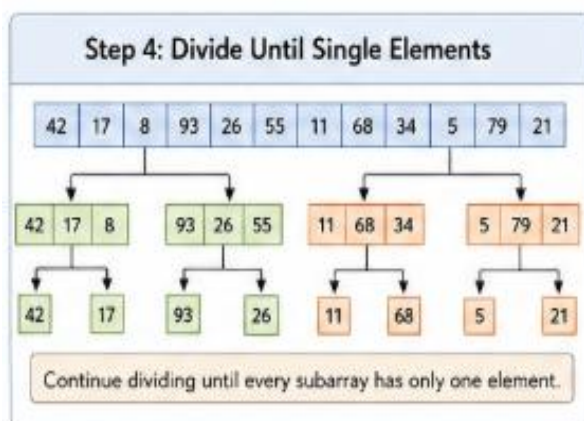
Eventually, every subarray contains one element.

Step 3 – Merge

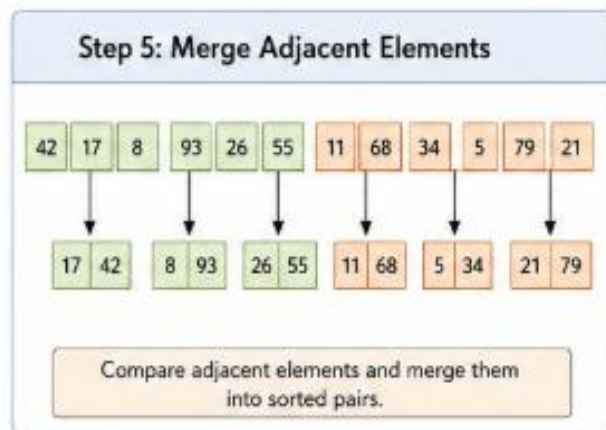


The individual elements are merged in sorted order.

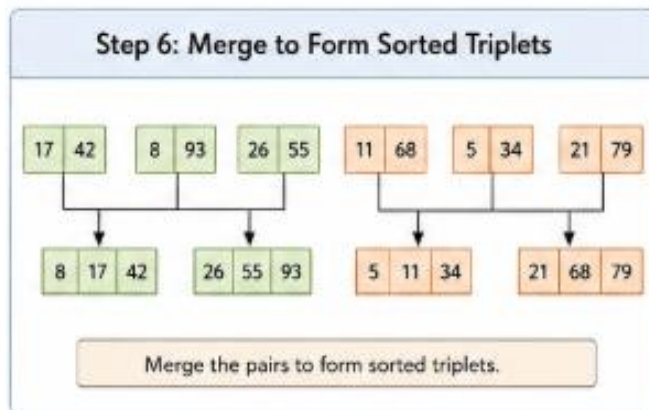
Step 4: Divide until Single Elements



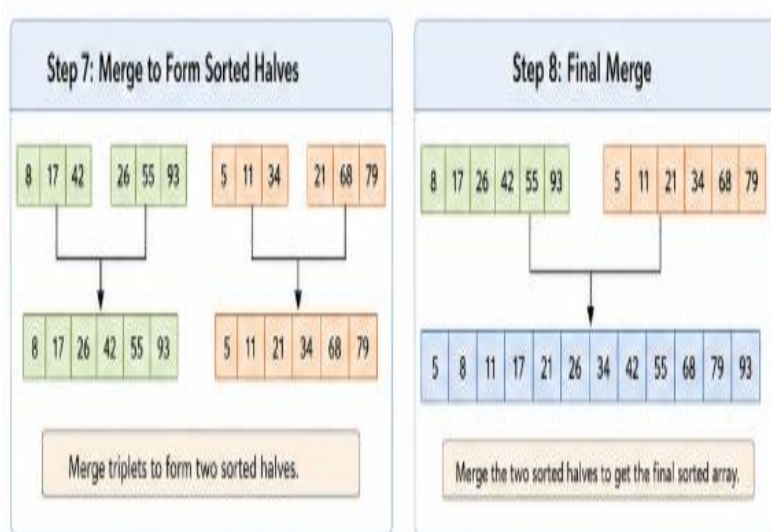
Step 5:



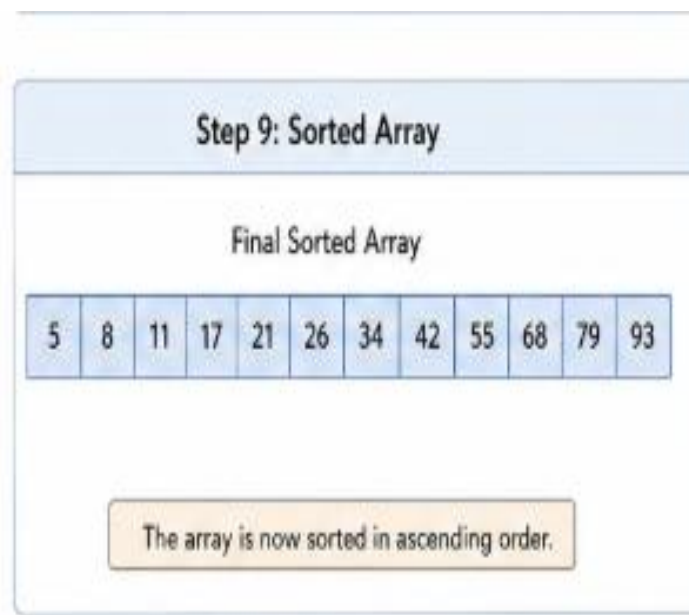
Step 6:



Step 7,8:



Final Sorted Sequence



{5, 8, 11, 17, 21, 26, 34, 42, 55, 68, 79, 93}

4. Merge Sort Pseudocode

MERGE_SORT(A, low, high)

1. If $low < high$:
2. $mid = (low + high) / 2$
- 3.
4. MERGE_SORT(A, low, mid)
5. MERGE_SORT(A, mid + 1, high)
6. MERGE(A, low, mid, high)

Merge Procedure

MERGE(A, low, mid, high)

1. Create temporary arrays L and R

2. Copy left half into L
3. Copy right half into R
4. Compare elements from L and R
5. Place smaller element into A
6. Continue until one array is empty
7. Copy remaining elements

5. Python Implementation

```
def merge_sort(arr):  
    if len(arr) <= 1:  
        return arr  
  
    mid = len(arr) // 2  
    left = merge_sort(arr[:mid])  
    right = merge_sort(arr[mid:])  
    return merge(left, right)  
  
def merge(left, right):  
    result = []  
    i = 0  
    j = 0  
    while i < len(left) and j < len(right):  
        if left[i] <= right[j]:  
            result.append(left[i])  
            i += 1  
        else:
```

```

        result.append(right[j])
        j += 1
    while i < len(left):
        result.append(left[i])
        i += 1
    while j < len(right):
        result.append(right[j])
        j += 1
    return result

# Given dataset
A = [42, 17, 8, 93, 26, 55, 11, 68, 34, 5, 79, 21]
print("Original Array:")
print(A)
sorted_array = merge_sort(A)
print("\nSorted Array:")
print(sorted_array)

```

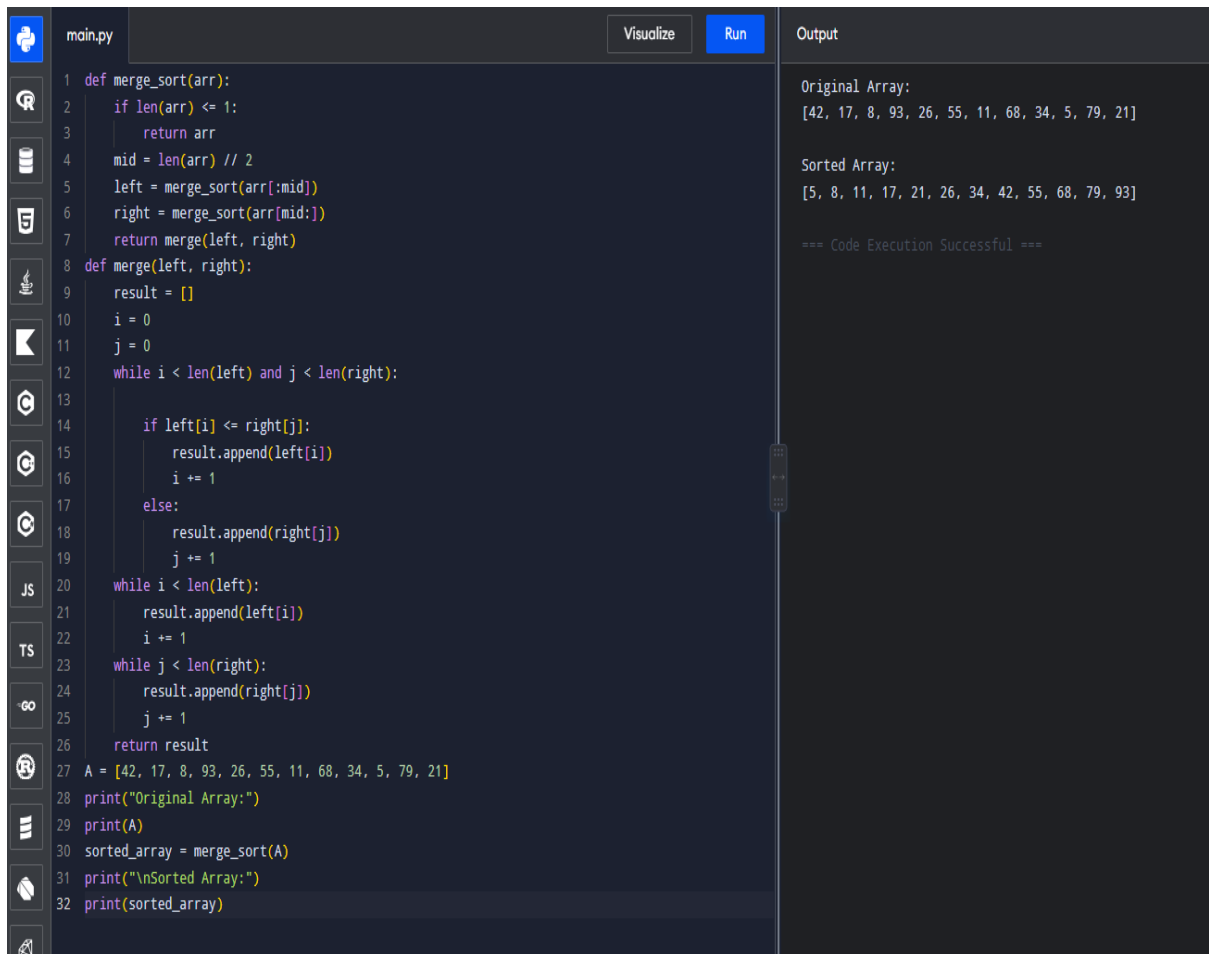
Expected Output

Original Array:

[42, 17, 8, 93, 26, 55, 11, 68, 34, 5, 79, 21]

Sorted Array:

[5, 8, 11, 17, 21, 26, 34, 42, 55, 68, 79, 93]



```
1 def merge_sort(arr):
2     if len(arr) <= 1:
3         return arr
4     mid = len(arr) // 2
5     left = merge_sort(arr[:mid])
6     right = merge_sort(arr[mid:])
7     return merge(left, right)
8
9 def merge(left, right):
10     result = []
11     i = 0
12     j = 0
13     while i < len(left) and j < len(right):
14         if left[i] <= right[j]:
15             result.append(left[i])
16             i += 1
17         else:
18             result.append(right[j])
19             j += 1
20     while i < len(left):
21         result.append(left[i])
22         i += 1
23     while j < len(right):
24         result.append(right[j])
25         j += 1
26     return result
27
28 A = [42, 17, 8, 93, 26, 55, 11, 68, 34, 5, 79, 21]
29 print("Original Array:")
30 print(A)
31 sorted_array = merge_sort(A)
32 print("\nSorted Array:")
33 print(sorted_array)
```

Output

Original Array:
[42, 17, 8, 93, 26, 55, 11, 68, 34, 5, 79, 21]

Sorted Array:
[5, 8, 11, 17, 21, 26, 34, 42, 55, 68, 79, 93]

=== Code Execution Successful ===

6. Complexity Analysis

Merge Sort divides the input into two approximately equal halves at every level.

The recurrence relation is:

$$T(n) = 2T(n/2) + O(n)$$

Therefore:

$$T(n) = O(n \log n)$$

Space Complexity

Merge Sort requires additional temporary arrays during merging.

$$\boxed{O(n)}$$

7. Comparison with Selection Sort

The assignment requires comparison between brute-force and efficient sorting approaches.

Feature	Selection Sort	Merge Sort
Strategy	Brute Force	Divide and Conquer
Best Case	$O(n^2)$	$O(n \log n)$
Average Case	$O(n^2)$	$O(n \log n)$
Worst Case	$O(n^2)$	$O(n \log n)$
Extra Space	$O(1)$	$O(n)$
Large Dataset	Slow	Efficient
Implementation	Simple	Moderately complex
Stability	Usually not stable	Stable
Main Operation	Find minimum	Compare and merge

8. Selection Sort Comparison

For $n = 12$:

$$\begin{aligned}\text{Comparisons} &= (n - 1) + (n - 2) + \dots + 1 \\ &= 11 + 10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 + 1\end{aligned}$$

$$\begin{aligned}
&= \frac{12(12 - 1)}{2} \\
&= \boxed{66}
\end{aligned}$$

- Thus, Selection Sort performs exactly 66 comparisons for the given 12-element input.
- The assignment explicitly requires this summation for Selection Sort.

9. Experimental Methodology

To evaluate scalability, the same dataset should be tested using different input sizes.

Suggested sizes:

1,000

10,000

100,000

1,000,000

For each size, measure:

- Execution time
- Number of comparisons
- Number of movements/operations
- Memory usage where applicable

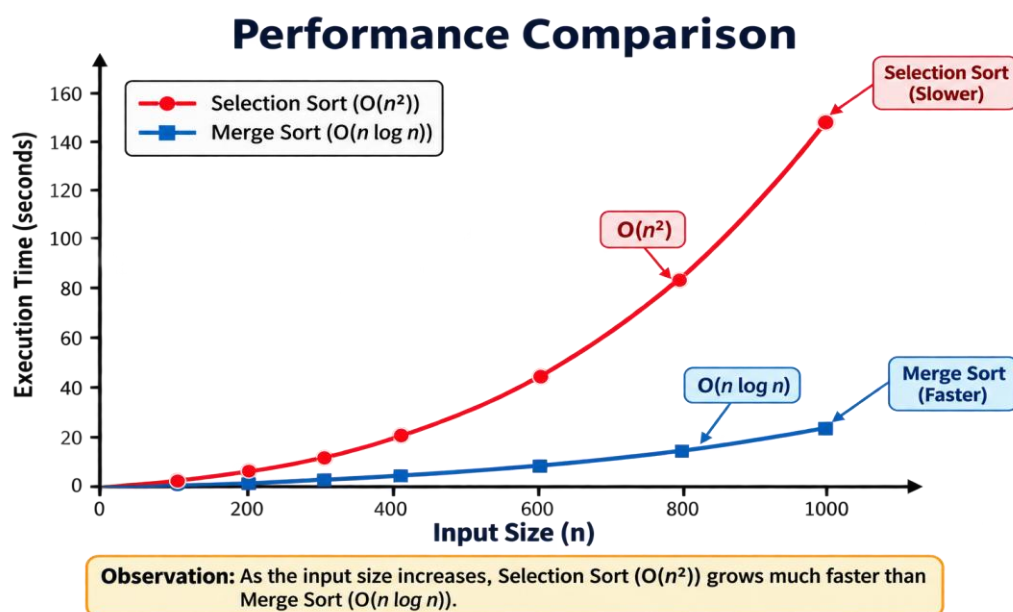
The assignment specifically requires datasets ranging from 10^3 to 10^6 records and comparison of Selection Sort, Merge Sort, and a hybrid sorting algorithm.

Important

Do not actually run Selection Sort on 1,000,000 elements unless your system can handle it, because its $O(n^2)$ behavior can make the experiment extremely slow.

10. Performance Comparison

Expected theoretical behavior:



Expected Observation

As the input size increases:

- Selection Sort's execution time increases rapidly.
- Merge Sort grows much more slowly.
- Merge Sort becomes increasingly advantageous for large datasets.
- The experimental results should approximately support the theoretical $O(n^2)$ and $O(n \log n)$ behaviors.

The assignment specifically asks students to determine whether experimental results support the expected complexity behavior.

11. Advantages of Merge Sort

1. Efficient for large datasets.
2. Guaranteed $O(n \log n)$ time complexity.
3. Performs consistently across best, average and worst cases.
4. Stable sorting algorithm.
5. Suitable for linked lists and external sorting.
6. Uses a systematic divide-and-conquer approach.

12. Limitations

1. Requires additional $O(n)$ memory.
2. More complex than Selection Sort.
3. For very small datasets, simpler algorithms may have lower overhead.
4. Recursive calls introduce additional computational overhead.

13. Engineering Decision

Merge Sort is selected as the efficient sorting algorithm because its $O(n \log n)$ complexity makes it more suitable for large datasets than Selection Sort's $O(n^2)$ complexity.

Although Selection Sort is easier to understand and requires very little additional memory, its quadratic growth makes it inefficient as the number of records increases.

Therefore:

Selection Sort is suitable for small datasets and educational demonstration of brute-force techniques, whereas Merge Sort is preferable for large-scale data processing.

14. Broader Considerations

Sustainability

Merge Sort can reduce computation time for large datasets compared with quadratic sorting algorithms. Lower execution time can contribute to reduced computational resource consumption.

Economic Impact

Faster sorting can improve the efficiency of data-processing applications and reduce processing costs for large datasets.

Accessibility

The algorithm can be implemented using freely available programming languages such as Python, making it accessible to students and developers.

Professional Responsibility

The implementation should be tested using appropriate datasets and the measured results should be reported honestly without artificially modifying performance values.

15. Student Reflection

What did I learn?

Through this assignment, I learned how the Divide-and-Conquer strategy can be used to solve sorting problems efficiently. I understood how Merge Sort recursively divides an array and combines sorted

portions. I also learned how theoretical complexity relates to actual execution time.

What would I improve?

If additional time and resources were available, I would test Merge Sort using larger real-world datasets and compare it with Quick Sort, Heap Sort and hybrid sorting algorithms. I would also create graphs showing execution time and operation counts for different input sizes.

16. Conclusion

Merge Sort provides an efficient solution for sorting large datasets using the Divide-and-Conquer technique. For the given dataset,

{42, 17, 8, 93, 26, 55, 11, 68, 34, 5, 79, 21}

the algorithm produces:

{5, 8, 11, 17, 21, 26, 34, 42, 55, 68, 79, 93}

The theoretical analysis shows that Merge Sort has $O(n \log n)$ time complexity in the best, average and worst cases, while Selection Sort requires $O(n^2)$ time.

Therefore, Merge Sort provides significantly better scalability for large datasets, while Selection Sort remains useful for demonstrating simple brute-force sorting and for small inputs.

17. References

You can use:

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, MIT Press.

2. E. Horowitz, S. Sahni and S. Rajasekaran, *Computer Algorithms*, Computer Science Press.
3. Python Software Foundation, *Python Documentation*.
4. Course material, CSA0603 – Design and Analysis of Algorithms.